ATM TRANSACTION SEQUENCE SIMULATION

Python Programming Project Report

Project Topic	ATM Transaction Sequence Simulation
Programming Language	Python 3
Core Concepts	Input validation, loops, conditionals, counters, arithmetic

Prepared as an educational project report with algorithm, flowchart, pseudocode, source code, test cases, and sample console output.

1. Problem Statement

Develop a Python program that simulates a sequence of ATM deposits and withdrawals. The program begins with an opening account balance and a maximum permitted withdrawal amount per transaction. For each transaction, it accepts a transaction type and amount, validates the input, updates the balance only when the transaction is valid, and rejects invalid operations. At the end, it displays a summary containing the number of successful and rejected transactions, total deposits, total withdrawals, and final account balance.

2. Objectives

• Maintain an accurate running account balance throughout a transaction sequence.

• Process deposits and withdrawals using a loop.

• Reject zero or negative transaction amounts and unknown transaction types.

• Prevent withdrawals that exceed the per-transaction limit or available balance.

• Track successful and rejected transactions separately.

• Display a clear transaction summary with monetary values formatted to two decimal places.

3. Assumptions

• The opening balance must be zero or greater.

• The per-transaction withdrawal limit must be greater than zero.

• The number of transactions must be a non-negative integer.

• Transaction types are entered as D for deposit or W for withdrawal; lowercase letters are accepted.

• Each transaction amount must be numeric and greater than zero.

• Deposits are not restricted by the withdrawal limit.

• This is an educational simulation only; it does not connect to a real bank or process real payments.

4. Algorithm

1. Start the program.

2. Read and validate the opening balance, withdrawal limit, and number of transactions.

3. Initialize deposit total, withdrawal total, successful count, and rejected count to zero.

4. Repeat for each transaction:

5. a. Read the transaction type and amount.

6. b. Reject the transaction if the amount is non-numeric, non-positive, or the type is invalid.

7. c. For a deposit, add the amount to the balance and deposit total.

8. d. For a withdrawal, check the withdrawal limit and available balance before subtracting.

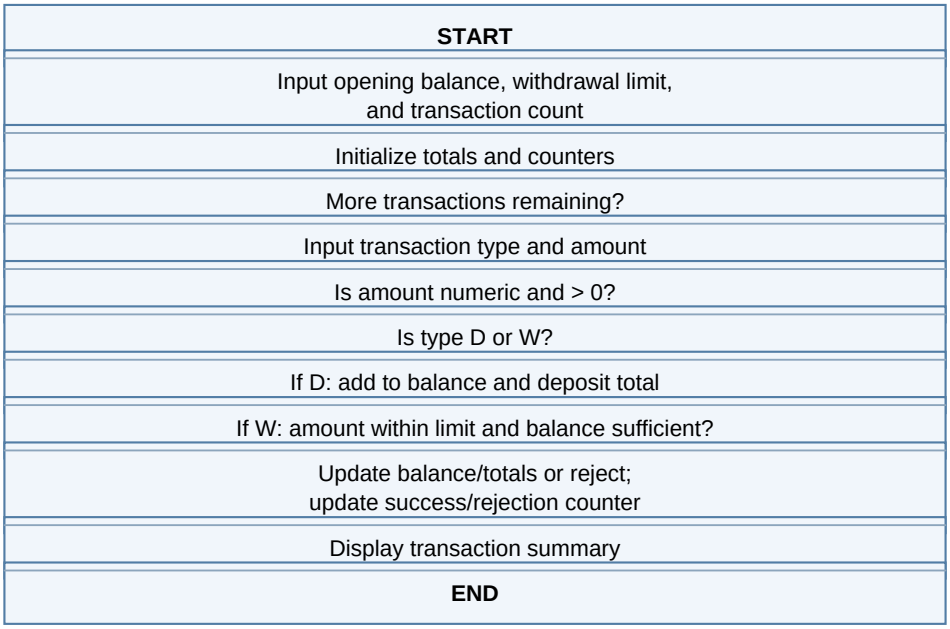
9. e. Update the success or rejection counter and display the result.

10. Display the transaction totals and final balance.

11. Stop.

5. Flowchart

The following flowchart outlines the main program logic. Decision branches marked Yes/No lead to the appropriate validation or transaction-processing step.



Flow: Start → Read setup → Initialize → Repeat transactions → Validate amount/type → Apply deposit or withdrawal rules → Update totals/counters → Show summary → End.

6. Pseudocode

```
BEGIN
    INPUT opening_balance
    IF opening_balance < 0 THEN
        DISPLAY error
        STOP
    ENDIF

    INPUT withdrawal_limit
    IF withdrawal_limit <= 0 THEN
        DISPLAY error
        STOP
    ENDIF

    INPUT transaction_count
    IF transaction_count < 0 THEN
        DISPLAY error
        STOP
    ENDIF

    deposits = 0
    withdrawals = 0
    successful = 0
    rejected = 0

    FOR each transaction from 1 to transaction_count
        INPUT transaction_type
        INPUT amount

        IF amount is not numeric OR amount <= 0 THEN
            rejected = rejected + 1
        ELSE IF transaction_type = "D" THEN
            balance = balance + amount
            deposits = deposits + amount
            successful = successful + 1
        ELSE IF transaction_type = "W" THEN
            IF amount > withdrawal_limit OR amount > balance THEN
                rejected = rejected + 1
            ELSE
                balance = balance - amount
                withdrawals = withdrawals + amount
                successful = successful + 1
```

```
            ENDIF
        ELSE
            rejected = rejected + 1
        ENDIF
    ENDFOR

    DISPLAY successful, rejected
    DISPLAY deposits, withdrawals, balance
END
```

7. Python Source Code

The program below uses input validation, a loop, conditional statements, and summary counters. Copy it into a file such as **atm_simulation.py** and run it with Python 3.

```
def atm_simulation():
    print("=== ATM Transaction Sequence Simulation ===")

    try:
        balance = float(input("Enter opening balance (Rs): "))
        if balance < 0:
            print("Opening balance cannot be negative.")
            return

        withdrawal_limit = float(input("Enter maximum withdrawal per transaction (Rs): "))
        if withdrawal_limit <= 0:
            print("Withdrawal limit must be greater than zero.")
            return

        n = int(input("Enter number of transactions: "))
        if n < 0:
            print("Number of transactions cannot be negative.")
            return

        total_deposits = 0.0
        total_withdrawals = 0.0
        successful = 0
        rejected = 0

        for i in range(1, n + 1):
            print(f"\\nTransaction {i}")
            kind = input("Enter type (D for deposit, W for withdrawal): ").strip().upper()

            try:
                amount = float(input("Enter amount (Rs): "))
            except ValueError:
                print("Rejected: amount must be numeric.")
                rejected += 1
                continue

            if amount <= 0:
                print("Rejected: amount must be greater than zero.")
                rejected += 1
            elif kind == "D":
                balance += amount
                total_deposits += amount
                successful += 1
                print(f"Deposit successful. Balance: Rs {balance:.2f}")
            elif kind == "W":
                if amount > withdrawal_limit:
                    print("Rejected: amount exceeds per-transaction withdrawal limit.")
                    rejected += 1
                elif amount > balance:
                    print("Rejected: insufficient balance.")
                    rejected += 1
                else:
                    balance -= amount
                    total_withdrawals += amount
                    successful += 1
                    print(f"Withdrawal successful. Balance: Rs {balance:.2f}")
            else:
                print("Rejected: invalid transaction type.")
                rejected += 1

        print("\\n=== Transaction Summary ===")
        print(f"Successful transactions: {successful}")
        print(f"Rejected transactions: {rejected}")
        print(f"Total deposits: Rs {total_deposits:.2f}")
        print(f"Total withdrawals: Rs {total_withdrawals:.2f}")
        print(f"Final balance: Rs {balance:.2f}")

if __name__ == "__main__":
    atm_simulation()
```

8. Test Cases

Test	Input / Scenario	Expected result
TC1	Deposit Rs 1,500 to Rs 5,000	Accepted; balance becomes Rs 6,500
TC2	Withdraw Rs 1,200; limit Rs 2,000; balance Rs 5,000	Accepted; balance becomes Rs 3,800
TC3	Withdraw Rs 2,500; limit Rs 2,000	Rejected: exceeds withdrawal limit
TC4	Withdraw Rs 6,000 from Rs 5,000; limit permits it	Rejected: insufficient balance
TC5	Amount is 0 or negative	Rejected: amount must be positive
TC6	Transaction type is X	Rejected: invalid transaction type
TC7	Amount entered as non-numeric text	Rejected: amount must be numeric
TC8	No transactions (count = 0)	Summary shown; final balance equals opening balance
TC9	Negative opening balance	Program displays error and stops
TC10	Withdrawal exactly equals balance and is within limit	Accepted; final balance becomes Rs 0.00

9. Sample Output (Console Screenshot Mock-up)

This is a sample console transcript showing a representative run. It is a formatted output example, not a screenshot captured from a live terminal. Run the source code to reproduce the output, then use your computer's screenshot shortcut if an actual screenshot is required for submission.

```
=== ATM Transaction Sequence Simulation ===
Enter opening balance (Rs): 5000
Enter maximum withdrawal per transaction (Rs): 2000
Enter number of transactions: 5

Transaction 1
Enter type (D for deposit, W for withdrawal): D
Enter amount (Rs): 1500
Deposit successful. Balance: Rs 6500.00

Transaction 2
Enter type (D for deposit, W for withdrawal): W
Enter amount (Rs): 1200
Withdrawal successful. Balance: Rs 5300.00

Transaction 3
Enter type (D for deposit, W for withdrawal): W
Enter amount (Rs): 2500
Rejected: amount exceeds per-transaction withdrawal limit.

Transaction 4
Enter type (D for deposit, W for withdrawal): W
Enter amount (Rs): 6000
Rejected: amount exceeds per-transaction withdrawal limit.

Transaction 5
Enter type (D for deposit, W for withdrawal): X
Enter amount (Rs): 100
Rejected: invalid transaction type.

=== Transaction Summary ===
Successful transactions: 2
Rejected transactions: 3
Total deposits: Rs 1500.00
Total withdrawals: Rs 1200.00
Final balance: Rs 5300.00
```

10. Conclusion

The ATM Transaction Sequence Simulation demonstrates how Python can process a series of deposits and withdrawals while maintaining a running balance. It validates transaction amounts, enforces a per-transaction withdrawal limit, prevents withdrawals beyond the available balance, and records both successful and rejected operations. The final summary gives a clear account of deposits, withdrawals, transaction counts, and the remaining balance. This project provides practice with loops, conditions, input validation, counters, and formatted output.

How to run the program

• Save the source code as atm_simulation.py.

• Open a terminal or command prompt in the folder where the file is saved.

• Run: python atm_simulation.py

• Enter the opening balance, withdrawal limit, number of transactions, and each transaction's details.